

工程评估 / YaoEngine

窗口库代码审查报告

从“能创建窗口”走向“可稳定复用的窗口模块”

审查对象：YaoEngine-core/src/core/Windows，以及紧密相关的 Event、Input、YaoEngine 主循环和 ImGui 接入。依据为 2026-09-09 本次审查末次核对的工作区，包含未提交改动；Git 基线 e37943739046。

核心判断

这套实现已经搭起了合理的雏形：用 YaoWindow 表达抽象接口，用 GLFWWindow 对接后端，用事件类把平台回调传给引擎。但目前仍存在影响正确性的缺口，尤其是初始化失败后的控制流、呈现时序，以及鼠标输入类型的一致性。建议先修复这些基础问题，再扩大窗口功能。

优先处理	为什么先做
初始化失败必须中止创建	创建失败仍继续使用窗口句柄；错误日志不能替代失败返回与资源回收。
拆开事件轮询与缓冲交换	当前 Clear → Poll/Swap → 图层更新的顺序不适合作为默认绘制循环。
统一鼠标按键定义	两个同名枚举冲突，事件侧按键值还与 GLFW 错位。

报告如何阅读

第 2 页是问题总览；第 3-8 页逐项解释触发条件、影响和修复方案；第 9 页给出接口与循环建议；第 10 页是实施顺序；第 11 页是验收与验证结果；第 12 页记录证据边界和资料来源。

本次结论来自源代码审查与隔离 C++ 探针。未启动完整引擎、未做显卡兼容或性能实测，因此“条件触发风险”不会写成已经发生的运行故障。

问题总览与现有结构

现有链路：YaoEngine 创建 YaoWindow → 工厂返回 GLFWWindow → GLFW 回调构造 Event → YaoEngine::OnEvent → UI / 实体图层。Input 和 ImGui 又直接取得 GLFW 原生窗口，形成并行的输入读取路径。

编号 / 级别	不足	性质
W01 · P1	初始化失败后继续运行	失败条件下的正确性问题
W02 · P1	交换缓冲发生在图层绘制前	当前调用顺序已确认
W03 · P1	鼠标枚举重复定义且数值错位	编译与语义问题
W04 · P1	持有原生句柄的对象可以被复制	复制用法下的所有权风险
W05 · P1	窗口析构同时终止整个 GLFW	多窗口用法下的破坏性风险
W06 · P2	键盘、字符、滚轮事件链未完成	当前能力缺口
W07 · P2	事件消费返回值被忽略	分发语义与调用者不一致
W08 · P2	轮询快照会漏掉短按边沿	两次采样之间的时序风险
W09 · P2	尺寸状态与像素尺寸没有统一	状态缺口；DPI 风险有条件
W10 · P2	窗口、渲染和后端边界不清	复用与扩展成本

P1：建议在继续新增功能前修复，包含当前错误和 API 已允许的高影响误用。P2：完成基础稳定性后处理。级别表达修复先后顺序，不代表问题已在你的机器上全部复现。

已有基础值得保留

工厂返回 unique_ptr 形式的 Scope；YaoWindow 有虚析构；回调通过窗口 user pointer 获取实例数据，并检查回调是否存在；ImGui 路径已分别查询窗口尺寸和 framebuffer 尺寸。这些结构可以继续沿用。

02 / 最直接影响窗口可用性的问题

先修复创建与呈现

W01 初始化失败后仍继续使用资源

P1 · 控制流已确认，实际触发取决于环境或参数

代码依据 [A] windows.cpp:21-36; 本地 GLFW src/window.c:1038-1046。

glfwInit() 返回失败时只写日志；glfwCreateWindow() 的结果没有判空，就传给 glfwSetWindowUserPointer()；GLAD 加载失败后也继续完成构造。指定 OpenGL 4.6、宽高为 0、资源耗尽或驱动不满足要求，都需要有可靠的失败出口。

若 GLFW 已初始化而窗口创建失败，本地 GLFW 的设置 user pointer 路径会断言非空，并在后续访问窗口字段。继续执行会导致断言终止或无效访问。GLAD 失败则可能把错误延迟到后续渲染调用。

建议：把创建当作完整事务：校验参数 → 初始化运行环境 → 创建窗口 → 绑定上下文 → 加载图形函数 → 注册回调。任一步失败都返回明确错误并释放已获得资源。C++17 可采用异常加 RAII，或显式结果类型；构造函数抛出时，不能指望该对象的析构函数负责清理。

W02 呈现时序早于图层绘制

P1 · 已按审查期间更新后的主循环复核

代码依据 [B] YaoEngine.cpp:58-73; [A] GLFWWindow.h:15-23。

```
// Current sequence
Clear();
OnUpdate(); // glfwPollEvents(); glfwSwapBuffers(...);
UpdateInput();
UpdateLayers();
```

当前已存在 OnUpdate() 调用，因此不能再判断为“没有事件循环”。问题在于，若图层在 OnUpdate 中绘制默认后缓冲，其结果可能在下一轮 Clear 中被清掉，再被交换出去。当前 Editor 未挂载绘制层，故这里确认的是时序缺陷，没有声称实测出黑屏。

建议：拆成 PollEvents() 与 Present()：帧首处理事件和输入，检查关闭请求，再更新、清屏、绘制，帧末 Present()。不要简单地把现有 OnUpdate 移到帧首，因为它同时负责交换。

鼠标类型需要统一

W03 重复枚举与按键映射错位

P1 · 真实头文件可直接验证

代码依据 [C] MouseEvent.h:4-11、15-20; Input.h:75-90; [A] windows.cpp:79-96。

Input.h 与 MouseEvent.h 在同一命名空间分别定义了 YaoEngine::MouseButton。同时包含这两个公开头文件会触发类型重定义。即使它们暂时只出现在不同翻译单元中，也不应让同一个类型名称承载两套不同定义。

实际按钮	GLFW / Input.h	MouseEvent.h 中同数值的含义
左键	0 / Left = 0	None = 0
右键	1 / Right = 1	ButtonLeft = 1
中键	2 / Middle = 2	ButtonRight = 2

窗口回调直接 `static_cast<MouseButton>(button)`，因此左键会被写为 `None`，右键会被写为 `ButtonLeft`。当前事件类又没有 `GetButton()`，外部难以读取或排查这一错误；回调参数 `mods` 也被丢弃，组合键点击无法从事件本身还原。

建议的最小统一方式

```
enum class MouseButton : int {
    None = -1,
    Left = 0, Right = 1, Middle = 2,
    Button4 = 3, Button5 = 4,
    Button6 = 5, Button7 = 6, Button8 = 7
};
// Keep this definition in one shared header.
// Event payload: button + action + modifiers.
```

把唯一的枚举定义放到独立输入类型头文件，让轮询 API 和事件 API 共用。映射可以采用显式转换函数，并对未知数值给出结果；同时暴露只读 `GetButton()` / `GetModifiers()`。

验收要覆盖两层：同一 .cpp 同时包含 Input.h 与 MouseEvent.h 能编译；左、右、中及扩展键经过窗口回调后，事件读取值与状态查询值一致。仅改名字或仅加 getter 都不够。

04 / 单窗口暂时掩盖的风险

资源所有权与多窗口

W04 持有窗口句柄的对象允许浅复制

P1 · 复制操作可用；当前主流程尚未看到实际复制

代码依据 [A] GLFWWindow.h:9-34; windows.cpp:31。

GLFWWindow 直接持有 GLFWwindow*，析构会销毁它，却没有禁用复制构造和复制赋值。复制后两个 C++ 对象指向同一个原生窗口；一个销毁，另一个就留下失效句柄。复制赋值还可能覆盖目标原有句柄。

同时，原生窗口 user pointer 指向原对象内部的 m_windowdata，浅复制不会自动改绑。因已声明析构函数，不能把“可由右值构造”的类型特征直接解释为实现了安全移动；右值也可能调用复制构造。

建议：先明确禁止 copy 和 move，继续用 Scope 转移指针所有权。确实需要移动对象时，再自定义转移句柄、清空源对象和重新绑定 user pointer 的逻辑，并验证回调指向。

```
GLFWWindow(const GLFWWindow&) = delete;
GLFWWindow& operator=(const GLFWWindow&) = delete;
GLFWWindow(GLFWWindow&&) = delete;
GLFWWindow& operator=(GLFWWindow&&) = delete;
```

W05 一个窗口析构会终止全部窗口

P1 · 至少两个窗口或外部共享 GLFW 时触发

代码依据 [A] GLFWWindow.h:13; windows.cpp:21; 官方资料 [R1]。

每个窗口构造都调用 glfwInit()，每个析构都调用 glfwTerminate()。GLFW 的初始化不是按窗口配对的引用计数，Terminate 会销毁剩余窗口及全局资源。因此创建 A、B 后销毁 A，B 的 C++ 对象还活着，原生资源却已失效。

建议：增加应用级 WindowSystem / GlfwRuntime，由它一次初始化、最后终止；窗口析构只销毁自己的句柄。当前引擎只创建一个主窗口，这项不是“单窗口退出一定崩溃”，而是窗口工厂可被多次调用时的生命周期缺陷。

线程约定也应在这一层落实：窗口创建、事件处理和销毁在主线程进行；回调中只记录关闭意图，等回调返回后再销毁窗口。若与外部 GLFW 用户共存，还应明确初始化与终止的归属。

事件有类型，还需要完整链路

W06 键盘、文本与滚轮没有送达上层

P2 · 当前实现缺失已确认

代码依据 [A] windows.cpp:38-103; [C] KeybroadEvent.h:4-10、MouseEvent.h:53-61; [D] imguiimportLayer.cpp:112-143。

窗口层只注册关闭、移动、尺寸、鼠标位置和鼠标按钮回调。没有 key、char、scroll、focus 回调；KeyEvent 仍是未完成的抽象类型。MouseScrolledEvent 还缺少 public:，构造和读取接口都不可从外部使用。

Input 可以轮询键盘状态，但按键状态不能替代 Unicode 文本输入，也不能恢复滚轮增量。现有 ImGui 接入只填写鼠标位置和三个按钮，未接入键盘、字符、滚轮，所以接入该 UI 层后，相关控件交互仍不完整。

建议：打通“GLFW 回调 → 规范化事件 → 引擎 / UI 消费”整条路径；键盘携带 key、scancode、action、mods，文本使用独立 Unicode codepoint。补焦点事件供应用暂停控制或重置输入逻辑。中文输入法组合、候选窗行为需要单独做平台验证，不能仅凭 char 回调宣称完整支持。

W07 handled 没有形成一致的消费约定

P2 · 回调返回值被忽略；图层遍历不主动停止

代码依据 [C] Event.h:29-35; [B] YaoEngine.cpp:43-54、75-82。

Dispatch() 会检查 handled，却丢弃 func() 的返回值。WindowClose() 返回 true 不会让事件变为已处理。外层 UI / 实体图层循环也不检查 handled，因此只靠某个图层“返回已处理”不能阻止后续 OnEvent 被调用。当前图层可以自行手动维护 handled，但这一约定未被接口落实。

```
if (!event.handled && event.GetEventType() == T::GetStaticType()) {
    event.handled = func(static_cast<T>(event));
    return true;    // means type matched and handler ran
}
return false;
```

建议：限定 handler 返回 bool，明确 true 表示消费；图层按期望优先级分发，并在 handled 后停止。若允许用户取消关闭，应先由应用处理 CloseRequested，再决定退出；消费事件和批准关闭是两个不同决策。

06 / 日常使用中容易漏掉的边界

输入边沿与尺寸语义

W08 只比较快照会遗漏短按

P2 · 按下和释放都发生在两次采样之间时触发

代码依据 [C] Input.cpp:34-45、65-94；官方资料 [R3]。

Input::OnUpdate() 保存上一帧快照后读取当前状态，再用两个快照推导 Down / Released。若一次按下和释放都在下一次采样前发生，当前和上一帧都可能是“未按下”，于是两个边沿都丢失。帧耗时较长或快速点击时更容易暴露。

建议：持续状态仍可用快照保存，边沿与文本 / 滚轮则由回调累积。每帧先清空本帧边沿，再处理系统事件，让按下和释放分别写入本帧标记或队列。短按同帧发生时，允许 down=true 与 released=true，同时 held=false。Sticky 模式只能缓解部分点击丢失，不能代替完整事件队列。

不要把每帧读取按键表直接定性为性能瓶颈。本次没有 CPU 测量，W08 的核心是输入语义与事件完整性。

W09 状态副本和像素尺寸不一致

P2 · 状态维护缺失已确认；DPI 影响依平台而定

代码依据 [A] windows.h:5-20、GLFWWindow.h:29-34、windows.cpp:18-20、59-68；[B] YaoEngine.cpp:79-81；[D] imguiimportLayer.cpp:117-130。

WindowData 只在构造时保存宽高，resize 回调没有更新它；基类又有独立、未同步的 m_windowprop。窗口接口没有只读尺寸查询，实际使用者只好绕过抽象直接调用 GLFW，导致多份状态来源并存。

引擎把 window-size 回调中的尺寸直接传入 glViewport。屏幕坐标与 framebuffer 像素在部分平台会不同，应使用 framebuffer 尺寸。Windows 常见 GLFW 路径是 1:1，本次没有证据证明你的 Windows 机器已经出现 DPI 渲染错误；这是一项可移植性与接口语义问题。

建议：分别提供 GetWindowSize()、GetFramebufferSize() 和 GetContentScale()，对应更新事件；创建后查询实际值，在回调中先更新缓存再通知。framebuffer 为 0 时跳过绘制，并用等待或限频策略避免最小化后空转。

现有 ImGui 代码已经分开查询两种尺寸并设置 DisplayFramebufferScale，这部分可以作为统一实现的起点，不能据此声称整个引擎都缺少 DPI 处理。

模块边界与工程化不足

W10 窗口职责混入渲染与具体后端

P2 · 当前耦合已确认；多窗口上下文错误有条件

代码依据 [A] windows.h:13-17、GLFWWindow.h:15-26、windows.cpp:25-33；[B] YaoEngine.cpp:19-20；[C] Input.cpp:25-26。

YaoWindow 不声明 SetEventCallback(), 引擎必须 static_cast 到 GLFWWindow 才能注册回调。Clear() 写死 OpenGL 清屏颜色与掩码, OnUpdate() 同时轮询事件和交换缓冲；原生句柄只用 void* 暴露。替换后端或独立复用, 上层仍离不开 GLFW 和 OpenGL。

创建每个窗口时会切换当前上下文, 但 Clear() 不保证属于本窗口的上下文当前有效。如果先创建 A 再创建 B, 随后调用 A.Clear(), 实际 OpenGL 命令会落在当前的 B 上。当前单窗口正常路径不受这一多窗口场景影响。

建议：在抽象接口中提供事件绑定、状态查询和关闭请求；由 WindowSystem 处理一次全局 PollEvents。由 GraphicsContext / Renderer 负责上下文激活、清屏和呈现。原生句柄作为显式平台互操作出口, 返回值应带类型或后端标识。

随后再处理的工程化事项

事项	现状与改进方向
事件载荷	MouseMovedEvent 将 double 坐标转成 int (MouseEvent.h:44-50) , 丢失小数；WindowMoveEvent 的坐标无 getter (WindowEvent.h:22-28) 。补一致的只读访问。
回调生命周期	Event 在回调栈上创建, 只能同步借用；文档应禁止保存 Event&, 异步使用需复制载荷。Event 析构非虚；若引入 unique_ptr<Event>, 应先补虚析构。
配置与诊断	OpenGL 4.6、1900×1900、清屏色写死；没有 swap interval 设置；成功创建同时输出 error 日志（windows.cpp:29-30）。暴露必要配置, 记录真实错误码与描述。
状态与头文件	标题保存为 const char*, 缓存缺少所有权；GLFW 会复制创建时的标题, 不能说调用后原生标题必然悬空。建议缓存用 std::string。统一 GLFWwindow.h / GLFWWindow.h 大小写。

建议的最小重构形态

无需立刻引入庞大的框架。先把全局运行环境、单个窗口与绘制上下文分开，当前 GLFW 后端仍然可以继续使用。下面是设计草案，并非已经写入项目的补丁。

模块	负责什么	生命周期
WindowSystem	GLFW 初始化 / 终止、主线程约束、事件轮询	先创建，最后销毁
Window	原生窗口、配置、尺寸 / 焦点、事件与关闭状态	拥有独占句柄
GraphicsContext	当前上下文、swap interval、呈现	渲染资源释放后再销毁
Renderer / Input	绘制与清屏 / 持续状态和本帧边沿	在明确帧阶段运行

帧循环示意

```
while (running) {
    input.BeginFrame();           // reset per-frame edges
    windowSystem.PollEvents();    // callbacks append input/events
    if (window.CloseAccepted()) break;

    if (window.FramebufferEmpty()) {
        windowSystem.WaitEventsTimeout(0.05);
        continue;
    }

    time.Update();
    context.MakeCurrent();
    UpdateLayers(time.Delta());
    renderer.Clear(clearColor);
    RenderLayers();
    context.Present();
}
```

如果仍保留“图层 OnUpdate 同时绘制”的约定，就把 Clear 放到图层调用之前，并保证 Present 在图层之后；不要求为了采用此顺序立即拆开所有图层。示例中的关闭、等待和上下文 API 需要新增。

先制定四条可检查的约定

- ① 对象不可复制，所有权只通过 Scope 转移。
- ② 生命周期与事件轮询在主线程执行。
- ③ 回调只同步借用事件，销毁动作延后。
- ④ 使用者仅通过明确接口区分逻辑尺寸与像素尺寸。

09 / 分成四个可验收的小阶段

改进顺序与完成标准

阶段	工作内容	完成标准
1 · 正确性	W01 创建失败处理；W02 帧阶段拆分；W03 按键定义统一。	失败创建不进入渲染；图层绘制出现在本帧；两个输入头能组合编译。
2 · 所有权	W04 禁止浅复制；W05 引入应用级运行环境；明确单 / 多窗口承诺。	编译器拒绝复制；销毁一个窗口不破坏其他窗口；错误路径不泄漏。
3 · 交互完整性	W06-08 补键盘 / 文本 / 滚轮链路、handled 约定和本帧边沿。	UI 与游戏层消费顺序可预测；短按、修饰键和文本输入有独立测试。
4 · 可复用性	W09-10 统一状态、隔离渲染职责，完善构建目标和文档。	可独立创建窗口而无需整个引擎；尺寸、上下文和原生句柄约定明确。

真正值得补的自动化验证

优先做接口组合编译、枚举映射、回调分发语义、边沿累积和失败回收验证。这些测试不依赖真实桌面，容易稳定运行。对于窗口、GPU 和 DPI 行为，再用小型演示程序执行人工或图形环境测试。

单独建立 WindowSmokeTest 可执行目标，只链接窗口、事件、GLFW 与必要的图形加载器；避免为了验证一个窗口，把 CUDA、Mono、场景和编辑器都带上。当前 core.lua 把它们聚合进引擎核心，仓库自研区域未检索到专门窗口测试。

暂时不必优先做

全屏、多显示器、无边框、图标、剪贴板、文件拖放、光标样式和更多后端，可以依据实际用途逐步加入。没有这些功能本身不说明库写错；应先确定库面向单窗口引擎，还是可独立复用的通用窗口层。

修复错误处理和所有权，并不要求立刻支持所有平台。即使暂时只支持 Windows + OpenGL，也应让这一限制在配置、文档和失败信息中清楚可见。

验证结果与回归清单

本次已完成

读取窗口、事件、输入、主循环和 UI 接入源码，并用 MSVC / C++17 对真实头文件编译运行隔离探针；未创建真实窗口，探针未启用 Tracy。结果如下。

验证项	观察结果
双头文件组合编译	MSVC 报 C2011: MouseButton 重定义；位置为 MouseEvent.h:4 / Input.h:75。
按键数值	真实头文件输出：GLFW 左键 = 0，事件侧 ButtonLeft = 1。
窗口所有权类型特征	copy_constructible=1, copy_assignable=1；由右值构造也可用，但不代表实现了安全移动。
分发器回调返回 true	匹配成功 = 1，调用后 handled = 0，证实消费结果未写回。
事件类型可用性	Event 虚析构 = 0；MouseScrolledEvent 从 float,float 公开构造 = 0。

修复后应验证的场景（本次未运行）

场景	期望结果
失败创建 / 不支持的上下文	返回可读错误；不注册空句柄回调、不进入绘制；已取得资源释放。
有图层绘制的单窗口	处理输入后绘制，再交换；每帧内容可见，点击关闭后停止后续更新。
快速短按 / 同帧按下释放	Down 和 Released 都被记录，最终 held 状态正确。
滚轮 / 组合键 / 文本	上下层获取一致载荷；消费后停止传播；验证 Unicode 与输入法体验。
双窗口 / 上下文切换	各窗口绘制到自身上下文；销毁 A 后 B 仍能更新与关闭。
缩放 / 跨屏 / 最小化	像素视口匹配 framebuffer；为 0 时不渲染；恢复后尺寸正确。

没有把探针编译成功等同于完整工程构建成功。没有运行完整引擎、真实双窗口或显卡测试，也未测帧率、CPU 使用率或内存泄漏。

代码索引与审查边界

代码索引

除另行说明，路径均相对项目根目录；行号对应本次审查核对的工作区。源文件继续编辑后，行号可能变化。

标记	位置
[A] 窗口层	YaoEngine-core/src/core/Windows/ windows.h、windows.cpp、GLFWWindow.h
[B] 引擎接入	YaoEngine-core/src/YaoEngine.cpp、YaoEngine.h YaoEngine-Editor/src/Editor.cpp
[C] 事件 / 输入	YaoEngine-core/src/core/Event/ Event.h、MouseEvent.h、WindowEvent.h、KeybroadEvent.h、Input.h、Input.cpp
[D] UI 接入	YaoEngine-core/src/core/Layer/imguiimportLayer.cpp
[E] 构建 / 依赖	YaoEngine-core/core.lua YaoEngine-core/Dep/GLFW/GLFW.lua、include/GLFW/glfw3.h、src/window.c

官方资料 (2026-09-09 查阅)

- [R1] [GLFW Introduction to the API](#): 初始化、终止、主线程及重入约束。
- [R2] [GLFW Context guide](#): 当前上下文与加载图形函数。
- [R3] [GLFW Input guide](#): 事件处理、轮询局限、字符及滚轮输入。
- [R4] [GLFW Window guide](#): 窗口尺寸、framebuffer 尺寸与缓冲交换。

结论边界

本地 glfw3.h 的版本宏为 3.5.0；这仅代表编译期头文件版本，未读取运行中的 GLFW 二进制版本。报告以自研源码为主要依据，官方文档用于核对稳定 API 约束，没有把依赖目录的版本宏解释为发布状态。

审查期间主循环文件发生更新，已按含 m_window->OnUpdate() 的新内容重核 W02，未将早先的“缺少调用”作为最终结论。没有修改库的源代码。本报告不覆盖整个引擎的场景、脚本、渲染算法或所有第三方依赖。

窗口原生资源的 RAII 雏形可以继续保留。最有价值的下一步是让创建失败、每帧呈现、事件消费和对象所有权都拥有清晰、可验证的契约。